

Basic R and how to read in data

This guide is partly based on online material from Amy Willis, Kiirsti Owen and Amelia McNamara, and the book “R for Data Science” by Hadley Wickham and Garrett Golemund. Thank you amazing R community!

R as a calculator

In the Console window below, type: 2+2 and press Enter Also try:

```
2^5
```

```
## [1] 32
```

```
3/10
```

```
## [1] 0.3
```

```
(3+5)^2
```

```
## [1] 64
```

```
sqrt(4)
```

```
## [1] 2
```

Tip: To run a line (or multiple lines) of code from a script without typing them into the Console, select the line(s) you want to run and press Ctrl+Enter (Command+Enter on a Mac)

Objects

R stores data as objects. You create new objects when you assign a value to them using “<-”:

```
x <- 3 # Check the "Environment" window!
```

Tip: use the R studio shortcut Alt+ - (Alt and the minus sign) to easily create the assignment symbol <-

```
y <- 6  
x+y
```

```
## [1] 9
```

Tip: R is case sensitive so if you’ve defined your object as x, it will not recognise (capital) X. Similarly, the function for square root is sqrt, R will give you an error if you try to use Sqrt.

Packages

Packages extend the functionality of base R. They are distributed via CRAN: the Comprehensive R Archive Network

To install a package, use: `install.packages("packagename")` You then need to load it, using `library(packagename)`

We will be using a collection of packages called the Tidyverse:

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.5.2
```

```
## Warning: package 'ggplot2' was built under R version 4.5.2
```

```
## Warning: package 'tibble' was built under R version 4.5.2
```

```
## Warning: package 'tidyr' was built under R version 4.5.2
```

```
## Warning: package 'readr' was built under R version 4.5.2
```

```
## Warning: package 'purrr' was built under R version 4.5.2
```

```
## Warning: package 'dplyr' was built under R version 4.5.2
```

```
## Warning: package 'forcats' was built under R version 4.5.2
```

```
## Warning: package 'lubridate' was built under R version 4.5.2
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.1.6
```

```
## v forcats    1.0.1      v stringr    1.5.2
```

```
## v ggplot2    4.0.1      v tibble     3.3.0
```

```
## v lubridate  1.9.4      v tidyr      1.3.1
```

```
## v purrr      1.2.0
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()    masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

When you load the tidyverse, you'll see a message about conflicts. As there is an (increasingly) large number of packages in R, it is possible to have functions with the same name in more than one package. The message tells you that packages dplyr and stats both have a function called filter and the one that will be used is the one from dplyr. It is the one that was loaded last.

If you want to use a function from a particular package, you need to include `packagename::` before the name of the function.

In this example, you can use `stats::filter()` instead of just `filter()` to use filter from the stats package.

```
find("filter") # this shows you the packages a function belongs to, in order of priority
```

```
## [1] "package:dplyr" "package:stats"
```

The tidyverse packages we will be using mostly in this course are readr (for reading in data), dplyr (for transforming data) and ggplot2 (for plotting).

Functions

When using the Tidyverse, you can call functions in two ways:

```
sqrt(4) # base R
```

```
## [1] 2
```

```
4 %>%  
  sqrt # "pipe" operator (you can read is as "and then...")
```

```
## [1] 2
```

Tip: use the R Studio shortcut Ctrl + Sft + M to create the pipe operator %>%

Tip: If you are not sure what a function does, type ?functionname in the Console, e.g. ?sqrt

Reading in data

Before we read in our data, let's consider where we have saved our data file. Since we want our code to be reusable (by us and other people), the last thing we want is to include the location of the file in our code, something like:

```
"C:/dimitra/data/datafile.csv"
```

The above would only work for me, and only for the particular computer where folder "dimitra" contains a folder called "data".

To avoid these issues, we need to do two things:

1. Use R projects. (I hope you are doing that already!) Save the data and R markdown file inside the R project. Exactly where you save your code doesn't matter, you just need to note the location of your data with respect to the .Rproj file.
2. Use the R package "here". "Here" points to the location of the .Rproj file (which is the working directory for your project), so you just need to add "here" in front of the relative path to your data file.

For example, if your data file (a comma-separated value (csv) file) was saved inside a "data" directory, you would say:

```
library(here)
```

```
fev_data <- read_csv(here("data/fev.csv"))
```

To read in a file that is saved in the same directory as the .Rproj file:

```
library(here)
```

```
## Warning: package 'here' was built under R version 4.5.2
```

```
## here() starts at C:/Users/abhih/OneDrive/Documents/GitHub/Rstudio-Practice
```

```
fev_data <- read_csv(here("fev.csv"))
```

```
## Rows: 654 Columns: 7
```

```
## -- Column specification -----  
## Delimiter: ","  
## dbl (7): seqnbr, subjid, age, fev, height, sex, smoke  
##  
## i Use 'spec()' to retrieve the full column specification for this data.  
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

(Remember to install the “here” package the first time.)

→ How would you use read_csv with the pipe operator?

```
# Add your code here!
```

Look at the top few rows of the data:

```
head(fev_data)
```

```
## # A tibble: 6 x 7  
##   seqnbr subjid   age   fev height   sex smoke  
##   <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  
## 1      1    301     9  1.71   57     0     0  
## 2      2    451     8  1.72  67.5     0     0  
## 3      3    501     7  1.72  54.5     0     0  
## 4      4    642     9  1.56   53     1     0  
## 5      5    901     9  1.90   57     1     0  
## 6      6   1701     8  2.34   61     0     0
```

fev_data is a tibble - this is a tidyverse structure similar to a data frame (from base R) but with some differences:

- default printing is shorter
- tells you the column types (character, double, etc.)
- doesn't change the types of inputs

Tip: if your data is in a Microsoft Excel spreadsheet, you will need a different package to read it in, such as readxl. So you'll need:

```
install.packages("readxl")
```

```
library(readxl)
```

```
excel_data <- read_xlsx(filename, sheet = 1) #(to read the first sheet)
```

→ How would you read in a text file? (Check the data import cheat sheet!) There is a text file in your dataset so you can practice: psa.txt

```
library(readr)
psa <- read_csv("psa.txt")
```

```
## Rows: 50 Columns: 1
## -- Column specification -----
## Delimiter: ","
## chr (1): ptid  nadirpsa pretxpsa ps      bss  grade    age    obstime...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
View(psa)
# Add your code here!
```

→ Have a look at the “Useful arguments” section of the data import cheat sheet. Use a few of them when you read in fev.csv and look at the data, is that what you expected?

```
# Add your code here!
```

→ Apply the summary function to a tibble. What does it do?

```
library(here)
fev_data <- read_csv(here("fev.csv"))
```

```
## Rows: 654 Columns: 7
## -- Column specification -----
## Delimiter: ","
## dbl (7): seqnbr, subjid, age, fev, height, sex, smoke
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
summary(fev_data)
```

```
##      seqnbr      subjid      age      fev
## Min.   : 1.0   Min.   : 201   Min.   : 3.000   Min.   :0.791
## 1st Qu.:164.2   1st Qu.:15811   1st Qu.: 8.000   1st Qu.:1.981
## Median :327.5   Median :36071   Median :10.000   Median :2.547
## Mean   :327.5   Mean   :37170   Mean   : 9.931   Mean   :2.637
## 3rd Qu.:490.8   3rd Qu.:53639   3rd Qu.:12.000   3rd Qu.:3.119
## Max.   :654.0   Max.   :90001   Max.   :19.000   Max.   :5.793
##      height      sex      smoke
## Min.   :46.00   Min.   :0.0000   Min.   :0.00000
## 1st Qu.:57.00   1st Qu.:0.0000   1st Qu.:0.00000
## Median :61.50   Median :1.0000   Median :0.00000
## Mean   :61.14   Mean   :0.5138   Mean   :0.09939
## 3rd Qu.:65.50   3rd Qu.:1.0000   3rd Qu.:0.00000
## Max.   :74.00   Max.   :1.0000   Max.   :1.00000
```

```
# Add your code here!
```

Operating on data: columns

Individual columns are identified using the \$ symbol:

```
head(fev_data$fev)
```

```
## [1] 1.708 1.724 1.720 1.558 1.895 2.336
```

```
summary(fev_data$fev)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    0.791   1.981   2.547   2.637   3.119   5.793
```

```
length(fev_data$fev)
```

```
## [1] 654
```

Other useful functions for tibbles and data frames:

```
names(fev_data)
```

```
## [1] "seqnbr" "subjid" "age"    "fev"    "height" "sex"    "smoke"
```

```
dim(fev_data)
```

```
## [1] 654    7
```

Other useful functions for columns:

```
max(fev_data$fev)
```

```
## [1] 5.793
```

```
mean(fev_data$fev)
```

```
## [1] 2.63678
```

```
sd(fev_data$fev)
```

```
## [1] 0.8670591
```